

- [Weaviate Workshop: Building a Vector Search Application](#)
 - [Learning Objectives](#)
 - [Deliverable](#)
 - [Challenges](#)
 - [Setup scenarios](#)
 - [Environment & tools](#)
 - [Why Streamlit?](#)
 - [Prerequisite](#)
 - [Readings](#)
 - [Data from MongoDB to Weaviate](#)
 - [Schema](#)
 - [Embeddings](#)
 - [FLOw](#)
 - [Streamlit app features](#)
 - [Deployment](#)
 - [streamlit cloud deployment details](#)
 - [Reranker](#)
 - [Quality Assessment](#)
 - [Share](#)

Weaviate Workshop: Building a Vector Search Application

Build and deploy a Streamlit application that performs semantic search using Weaviate

Learning Objectives

By the end of this workshop, students will be able to: - Understand vector databases and their use cases - Set up and configure a Weaviate instance - Import data from MongoDB to Weaviate with embeddings - Build a semantic search interface using Streamlit - Implement reranking to improve search results - Deploy their application on Streamlit Cloud

Deliverable

Add the github repo or the streamlit app URL in the google spreadsheet in the "Weaviate Search Engine" column

Challenges

The challenge of this workshop is to have an app ready in 3 hours!

You can only achieve that velocity if you work with a decent AI coding platform. I suggest windsurf which is much more complete than vscode with copilot. Cursor, Claude code (by far the best at the moment) or [Manus](#) are also great.

If you have never worked with these platforms now is the time to make the jump!

Setup scenarios

After the 1st project, you should have a dataset available in MongoDB, either local or on Atlas.

If that's the case, you import it from MongoDB into Weaviate.

Else:

- Use the IMDB Movies dataset from MongoDB Atlas sample datasets
 - if your dataset does not contain text,
 - if you do not have a dataset available on MongoDB

In all cases, you must (re)create the embeddings.

Environment & tools

- work on your local in python.
- you **have to** use an LLM to generate the code

You should use windsurf or claude code. game changer. Copilot is autocomplete on steroids whereas Windsurf is an experienced code engineer with access to the whole codebase.

Other AI coding platforms such as cursor, or claude code are fantastic.

Why Streamlit?

Streamlit is a simplified framework to publish data oriented websites with a few lines of python. Fast learning curve.

When you commit your code to a public github repo, streamlit can host it for free!

If you don't know streamlit yet, check out the [playground](#)

Prerequisite

- python running on local (conda, ...)
- A github account. you will need to create a public repo.
- A weaviate account and cloud cluster. (WCS)
- A mongoDB account, cluster and database

Readings

- <https://weaviate.io/developers/weaviate/quickstart>
- <https://weaviate.io/developers/weaviate/starter-guides/custom-vectors>
- <https://weaviate.io/developers/weaviate/starter-guides/managing-collections>

Data from MongoDB to Weaviate

The simple solution is to download your dataset from MongoDB as json and then to upload / import it to weaviate.

But there's also several ways to stream data directly from MongoDB Atlas to Weaviate without intermediate files:

Ask your AI to write a Python script that connects to both databases simultaneously and streams data.

Schema

You can import data into Weaviate without creating a schema. Weaviate will use all default settings, and infer what data type you use.

Or you can define a schema which is advised to optimize Weaviate's performance.

- <https://weaviate-docusaurus.vercel.app/developers/weaviate/getting-started/schema>

Embeddings

The simplest and costs free way to create embeddings is to use SentenceTransformer from Huggingface

```
from sentence_transformers import SentenceTransformer
model = SentenceTransformer('all-MiniLM-L6-v2')
```

Another cost free way is to use Ollama on local with the [nomic-embed-text](#) model. [Download Ollama](#). That would be my favorite solution. However, when you publish the app, you need a way to get embeddings for the query. Setting up Ollama to work remotely with an API endpoint is too complex for this workshop. So we'll stick with `SentenceTransformer`.

Flow

- create / connect to MongoDB, weaviate, create weaviate collection etc
- create public github repo for your code

Then - generate script that imports data from MongoDB to Weaviate - setup streamlit app.py - test on local - setup the deploy when pushing on github

Streamlit app features

- The user can input a text query.
- The app returns the N most meaningful texts related to that query along with similarity scores

In a left sidebar, the user can also select:

- number of documents to display OR similarity metric threshold value
- similarity netrics : cosine, dot product, ...
- different search algorithms : hybrid, BM25F, vector search
- drop downs for categorical fields in the dataset

You are absolutely free to add the features you like.

Deployment

the GitHub repository should contain - `app.py` (main Streamlit application) - `requirements.txt` (Python dependencies) - `README.md` (project documentation) - `config.py` (configuration management) - `.streamlit/secrets.toml` (for sensitive data)

Create a requirements.txt file with:

```
pip freeze > requirements.txt
```

To deploy on streamlit cloud:

Create a streamlit account on <https://streamlit.io/cloud>

- Connect GitHub repository
- Configure secrets for Weaviate credentials
- Deploy with one click

streamlit cloud deployment details

Make sure your app runs locally: Before deploying, test your app one last time locally to catch any last-minute errors.

Commit and Push: Commit all your changes (including app.py, requirements.txt, and any other necessary files) to your GitHub repository.

1. Connect to Streamlit Cloud:

Log In: Go to <https://streamlit.io/cloud> and log in using your GitHub account.

1. Create a New Application:

Click "New App": On your Streamlit Cloud dashboard, look for a button or link labeled "New App" or something similar. Click it.

1. Configure Your Deployment:

Repository: A dropdown menu will appear, allowing you to select the GitHub repository containing your Streamlit app. Find your repository in the list and select it. You may need to grant Streamlit Cloud permission to access your repositories if this is your first time.

Branch: Select the branch from which you want to deploy.

Main File Path: Specify the path to your main Streamlit application file (usually app.py). If your app.py file is in the root directory of your repository, you can simply enter app.py. If it's in a subdirectory, specify the full path (e.g., src/app.py).

1. Advanced Settings (Optional): Streamlit Cloud offers some advanced configuration options, but for most basic deployments, you can leave these at their default values. These options might include:

- Secrets Management: Allows you to securely store API keys or other sensitive information (covered in a separate guide).

1. Deploy!

Click "Deploy!": Once you've configured everything, click the "Deploy!" button.

Reranker

Reranking seeks to improve search relevance by reordering the result set returned by a search with a different model.

see <https://weaviate.io/developers/weaviate/concepts/reranking>

Note: the Hugging face reranker module requires a hugging face API key. Not sure if they offer a free tier. Use models like `cross-encoder/ms-marco-MiniLM-L-6-v2`

Quality Assessment

Test your application with:

- Different query types (exact match, semantic similarity, conceptual queries)
- Edge cases (empty queries, very long queries)
- Some kind of performance evaluation

Share

Once you have deployed your app, share the link in the discord and in the spreadshhet.

Don't forget to add your name in the app so I know who did it

First one to finish gets a high five!